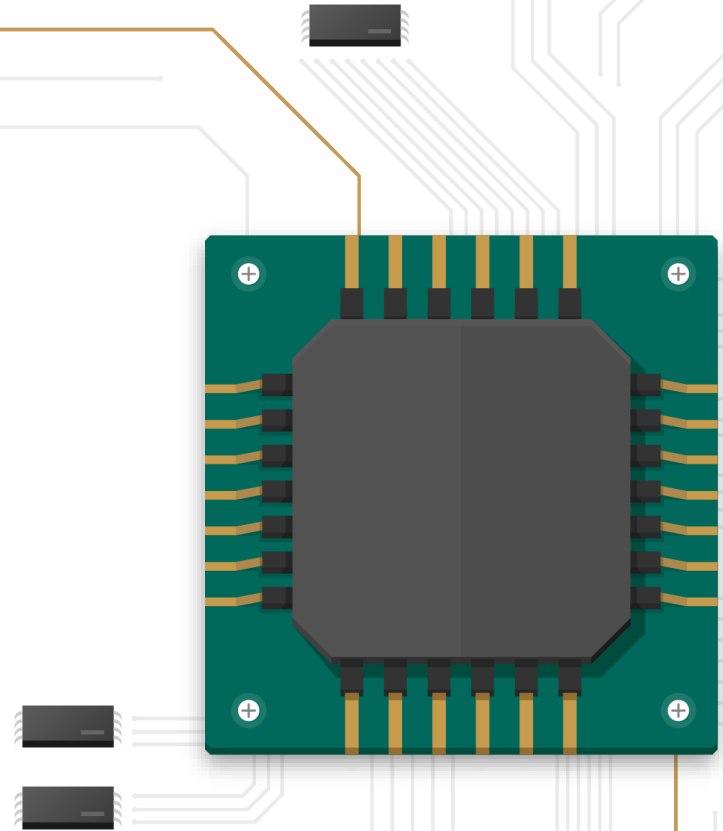




MULTIPLIER

روزبه سیدنوزادی - فرزاد بهارآرا
استاد راهنما : دکتر سید عابد حسینی



انواع الگوریتم های ضرب کننده

Array Multiplier.1

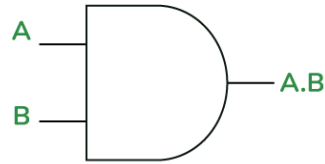
Shift and Add.2

Booths Algorithms.3

Wallace Tree Algorithms.4

AND GATE

2- Input AND Gate



Truth Table

A (Input 1)	B (Input 2)	X = (A.B)
0	0	0
0	1	0
1	0	0
1	1	1

نحوه عملکرد ضرب دودویی

در این روش محاسبه جواب مشابه ضرب دهدهی است

$$\begin{array}{rcccccccc} & & 1 & 1 & 0 & 0 & \longrightarrow & \text{Multiplicand} \\ \times & 1 & 1 & 0 & 1 & \longrightarrow & \text{Multiplier} \\ \hline & & 1 & 1 & 0 & 0 & \longrightarrow & \text{Partial Products} \\ & & 0 & 0 & 0 & 0 & \times & \longrightarrow & \text{Partial Products} \\ & 1 & 1 & 0 & 0 & \times & \times & \longrightarrow & \text{Partial Products} \\ & 1 & 1 & 0 & 0 & \times & \times & \times & \longrightarrow & \text{Partial Products} \\ \hline \text{Ans.} & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 \end{array}$$

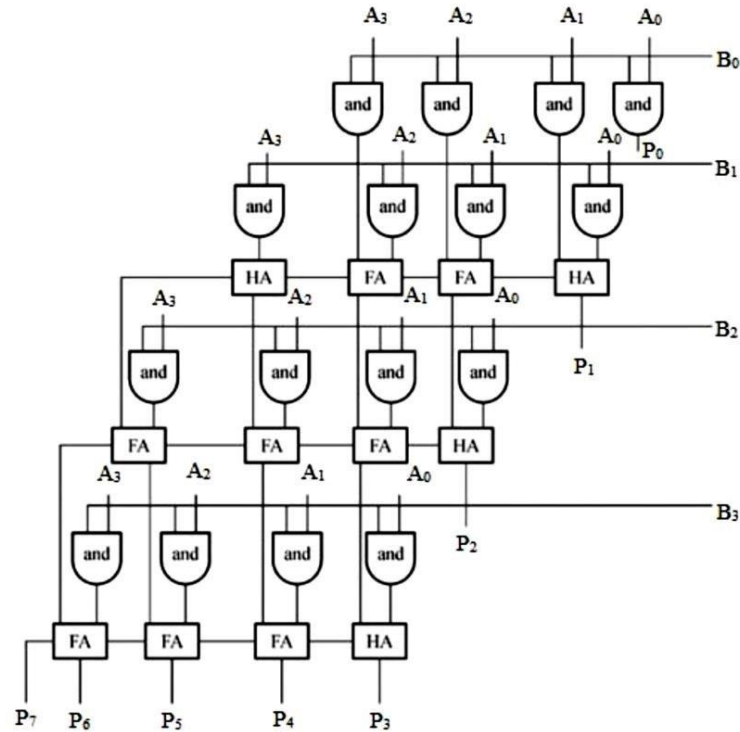
شماتیک روش Array Multiplier

تعداد گیت های مورد نیاز:

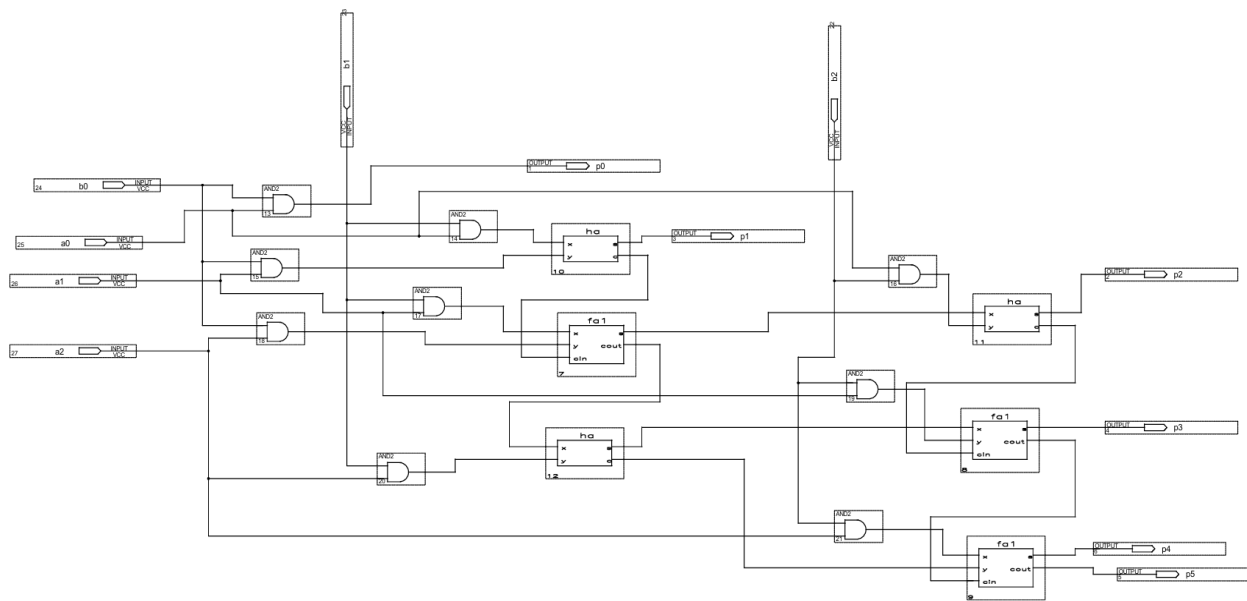
And Gates : n^2

HalfAdders : n

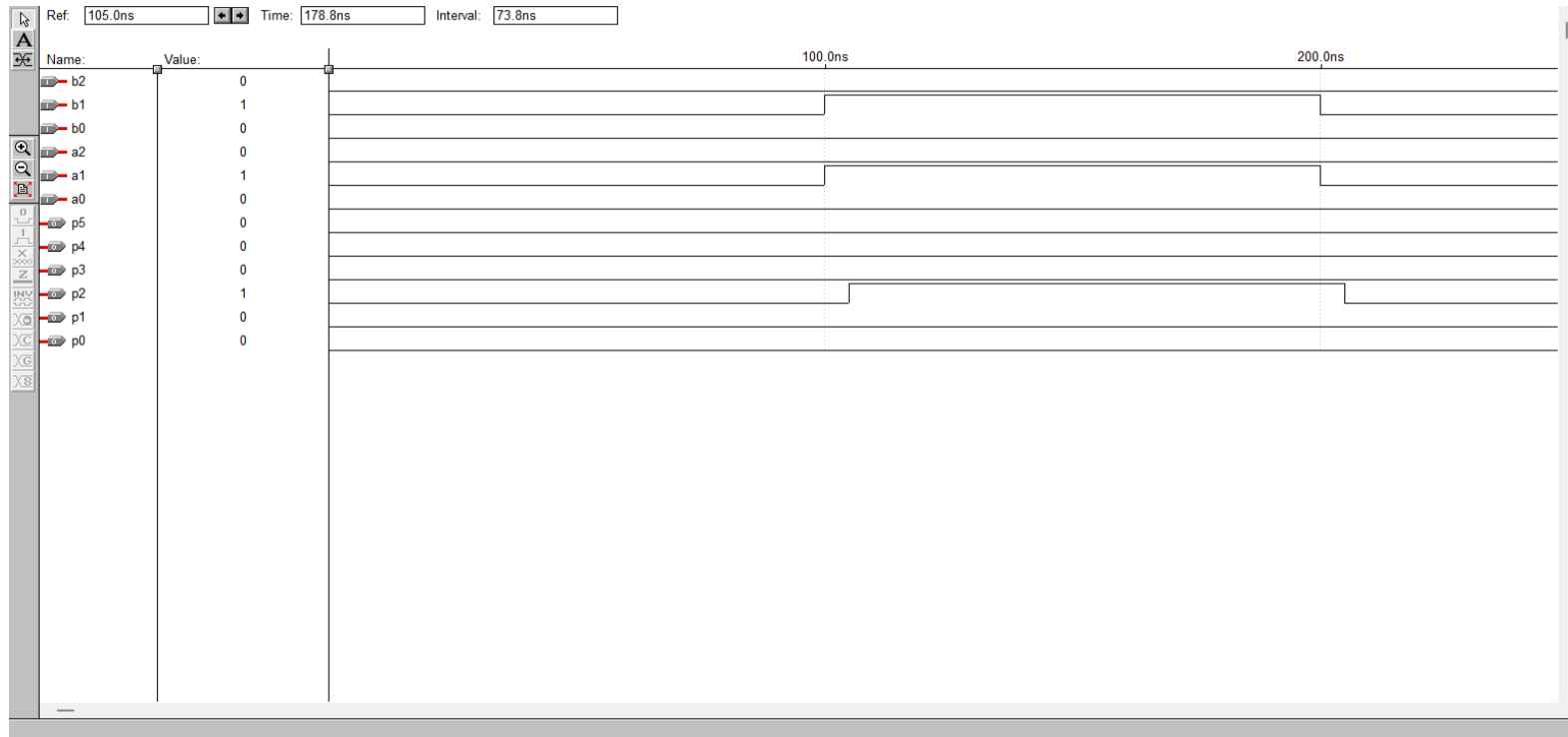
FullAdders : $n(n-2)$



شماتیک روش Array Multiplier



تاخیر در روش Array Multiplier



VHDL ك

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity multiplier4bit is
port (
    A: in STD_LOGIC_VECTOR (3 downto 0);
    B: in STD_LOGIC_VECTOR (3 downto 0);
    S: out STD_LOGIC_VECTOR (7 downto 0)
);
end multiplier4bit;

architecture Behavioral of multiplier4bit is
    -- Intermediate signal declarations (as in the original code)
    signal AB0, AB1, AB2, AB3: STD_LOGIC_VECTOR (3 downto 0);
    signal C1, C2, C3: STD_LOGIC_VECTOR (3 downto 0);
    signal P1, P2, P3: STD_LOGIC_VECTOR (3 downto 0);
```


VHDL كد

```
begin
```

```
-- --- Part 1: Partial Product Generation (AND gates) ---  
-- This part remains identical to your original code.
```

```
AB0(0) <= A(0) and B(0);  
AB0(1) <= A(1) and B(0);  
AB0(2) <= A(2) and B(0);  
AB0(3) <= A(3) and B(0);
```

```
AB1(0) <= A(0) and B(1);  
AB1(1) <= A(1) and B(1);  
AB1(2) <= A(2) and B(1);  
AB1(3) <= A(3) and B(1);
```

```
AB2(0) <= A(0) and B(2);  
AB2(1) <= A(1) and B(2);  
AB2(2) <= A(2) and B(2);
```

VHDL كد

```
-- HA1: half_adder_vhdl_code port map( AB0(1), AB1(0), C1(0), P1(0));
P1(0) <= AB0(1) xor AB1(0);      -- S
C1(0) <= AB0(1) and AB1(0);      -- Cout

-- FA1: full_adder_vhdl_code port map( AB0(2), AB1(1), C1(0), C1(1), P1(1));
P1(1) <= AB0(2) xor AB1(1) xor C1(0);      -- S
C1(1) <= (AB0(2) and AB1(1)) or (C1(0) and AB0(2)) or (C1(0) and AB1(1)); -- Cout

-- FA2: full_adder_vhdl_code port map( AB0(3), AB1(2), C1(1), C1(2), P1(2));
P1(2) <= AB0(3) xor AB1(2) xor C1(1);      -- S
C1(2) <= (AB0(3) and AB1(2)) or (C1(1) and AB0(3)) or (C1(1) and AB1(2)); -- Cout

-- HA2: half_adder_vhdl_code port map( AB1(3), C1(2), C1(3), P1(3));
P1(3) <= AB1(3) xor C1(2);      -- S
C1(3) <= AB1(3) and C1(2);      -- Cout

-- HA3: half_adder_vhdl_code port map( P1(1), AB2(0), C2(0), P2(0));
P2(0) <= P1(1) xor AB2(0);      -- S
C2(0) <= P1(1) and AB2(0);      -- Cout

-- FA3: full_adder_vhdl_code port map( P1(2), AB2(1), C2(0), C2(1), P2(1));
P2(1) <= P1(2) xor AB2(1) xor C2(0);      -- S
C2(1) <= (P1(2) and AB2(1)) or (C2(0) and P1(2)) or (C2(0) and AB2(1)); -- Cout
```

VHDL كد

```
-- FA4: full_adder_vhdl_code port map( P1(3), AB2(2), C2(1), C2(2), P2(2));
P2(2) <= P1(3) xor AB2(2) xor C2(1);    -- S
C2(2) <= (P1(3) and AB2(2)) or (C2(1) and P1(3)) or (C2(1) and AB2(2)); -- Cout

-- FA5: full_adder_vhdl_code port map( C1(3), AB2(3), C2(2), C2(3), P2(3));
P2(3) <= C1(3) xor AB2(3) xor C2(2);    -- S
C2(3) <= (C1(3) and AB2(3)) or (C2(2) and C1(3)) or (C2(2) and AB2(3)); -- Cout

-- HA4: half_adder_vhdl_code port map( P2(1), AB3(0), C3(0), P3(0));
P3(0) <= P2(1) xor AB3(0);              -- S
C3(0) <= P2(1) and AB3(0);              -- Cout

-- FA6: full_adder_vhdl_code port map( P2(2), AB3(1), C3(0), C3(1), P3(1));
P3(1) <= P2(2) xor AB3(1) xor C3(0);    -- S
C3(1) <= (P2(2) and AB3(1)) or (C3(0) and P2(2)) or (C3(0) and AB3(1)); -- Cout

-- FA7: full_adder_vhdl_code port map( P2(3), AB3(2), C3(1), C3(2), P3(2));
P3(2) <= P2(3) xor AB3(2) xor C3(1);    -- S
C3(2) <= (P2(3) and AB3(2)) or (C3(1) and P2(3)) or (C3(1) and AB3(2)); -- Cout

-- FA8: full_adder_vhdl_code port map( C2(3), AB3(3), C3(2), C3(3), P3(3));
P3(3) <= C2(3) xor AB3(3) xor C3(2);    -- S
C3(3) <= (C2(3) and AB3(3)) or (C3(2) and C2(3)) or (C3(2) and AB3(3)); -- Cout

S(0) <= AB0(0);
S(1) <= P1(0);
S(2) <= P2(0);
S(3) <= P3(0);
S(4) <= P3(1);
S(5) <= P3(2);
S(6) <= P3(3);
S(7) <= C3(3);

end Behavioral;
```

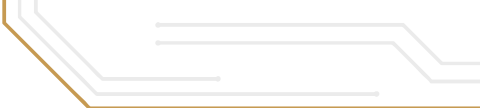
VHDL TESTBENCH

```
35
36  -- Add your stimulus here ...
37  process
38  begin
39      wait for 100ns;
40      A<="1000";
41      B<="1000";
42      wait for 100ns;
43      A<="0010";
44      B<="0100";
45      wait for 100ns;
46      A<="1001";
47      B<="0011";
48      wait;
49  end process;
50
51 end TB_ARCHITECTURE;
```

Signal name	Value	80	160	240	320	400	480
 nr A	9	U	8	2	9		
 nr B	3	U	8	4	3		
 nr S	1B	UU	40	08	1B		

ADD and SHIFT

با توجه به مقدار کم ارزش ترین بیت ضرب کننده مقدار ضرب شونده محاسبه و جمع می شود. در هر سیکل کلاک ضرب کننده یک بیت به سمت راست شیفت می شود و مقدارش مورد بررسی قرار می گیرد و اگر برابر صفر باشد فقط عملیات شیفت صورت می گیرد و اگر برابر یک باشد ضرب شونده با ثبات آکمولاتور جمع می شود سپس یک بیت شیفت به سمت راست رخ می دهد. و این فرآیند برای تمام بیت ها ضرب کننده تکرار می شود و عدد حاصل در آکمولاتور ذخیره می شود.



در این روش به طور چشمگیری نیاز به سخت افزار کاهش یافته است
از سخت افزار های مورد نیاز می توان به چند مورد زیر اشاره کرد:

۱. سه عدد ثبات

۲. شمارنده : شمارش تا n بیت بعد از شیفت

۳. واحد محاسبه (alu)

شماٹیک Add and Shift

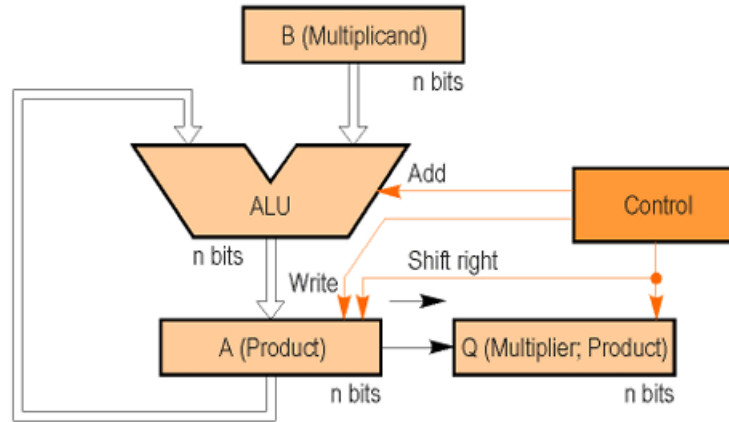


Figure 3.13. Final version of the multiplier circuit.

ماشین حالت

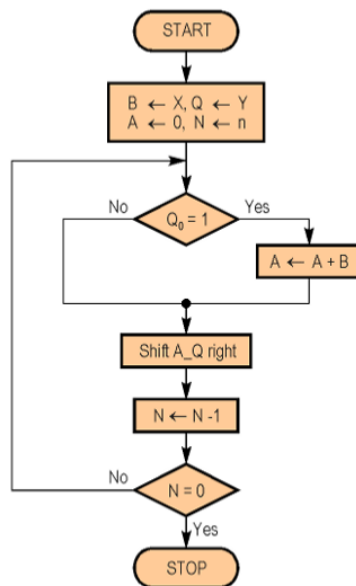


Figure 3.14. The final version of the multiplication algorithm.

VHDL كد

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity sequential_multiplier is
6   generic ( N : integer := 8 );
7   port (
8     clk      : in  std_logic;
9     reset    : in  std_logic;
10    start     : in  std_logic;
11    A         : in  std_logic_vector(N-1 downto 0);
12    B         : in  std_logic_vector(N-1 downto 0);
13    P         : out std_logic_vector(2*N-1 downto 0);
14    done      : out std_logic
15  );
16 end entity;
17
18 architecture rtl of sequential_multiplier is
19
20   type state_t is (IDLE, LOAD, CHECK_ADD, SHIFT, FINISH);
21   signal state : state_t := IDLE;
22
23   signal md_reg : unsigned(N-1 downto 0);
24   signal mq_reg : unsigned(N-1 downto 0);
25   signal acc_reg : unsigned(2*N-1 downto 0);
26   signal cnt     : integer range 0 to N;
27
28   -- Combined register for shifting
29   signal combined : unsigned(3*N-1 downto 0);
```

کد VHDL

```
31 begin
32
33 process(clk, reset)
34 begin
35     if reset = '1' then
36         state <= IDLE;
37         md_reg <= (others => '0');
38         mq_reg <= (others => '0');
39         acc_reg <= (others => '0');
40         combined <= (others => '0');
41         cnt <= 0;
42         done <= '0';
43         P <= (others => '0');
44
45     elsif clk'event and clk='1' then
46
47         case state is
48
49             -----
50             when IDLE =>
51                 -----
52                 done <= '0';
53                 if start = '1' then
54                     state <= LOAD;
55                 end if;
56
```

VHDL کد

```
when LOAD =>
```

```
md_reg  <= unsigned(A);  
mq_reg  <= unsigned(B);  
acc_reg <= (others => '0');  
cnt     <= 0;  
state   <= CHECK_ADD;
```

```
when CHECK_ADD =>
```

```
if mq_reg(0) = '1' then  
    acc_reg <= acc_reg + resize(md_reg, 2*N);  
end if;  
state <= SHIFT;
```

```
when SHIFT =>
```

```
-- Form ACC || MQ in one big vector  
combined <= acc_reg & mq_reg;  
  
-- Perform logical right shift by 1  
combined <= combined(3*N-1 downto 1) & '0';  
  
-- Decompose after shift  
acc_reg <= combined(3*N-1 downto N);  
mq_reg  <= combined(N-1 downto 0);
```

کد VHDL

```
86
87     cnt <= cnt + 1;
88
89     if cnt + 1 >= N then
90         state <= FINISH;
91     else
92         state <= CHECK_ADD;
93     end if;
94
95     -----
96     when FINISH =>
97     -----
98         P    <= std_logic_vector(acc_reg & mq_reg);
99         done <= '1';
100
101         if start = '0' then
102             state <= IDLE;
103             done <= '0';
104         end if;
105
106     end case;
107
108 end if;
109 end process;
110
111 end rtl;
112
```

نگاهی به سایر الگوریتم ها

Booths Algorithms

در این الگوریتم محاسبه حاصل ضرب برای اعداد منفی و مثبت به صورت یکسان است و کاهش تعداد ضرب های ضرب شونده اتفاق می افتد

Comparison of Booth and shift and add methods

```
Multiplicand 010101
Multiplier   001010
             000000
             010101
             000000
             010101
             000000
             000000
             000000
000011010010 Result
```

REGULAR SHIFT AND ADD MULTIPLICATION

```
Multiplicand 010101
Multiplier   001010
             111
000000001010 Intermediate
0000001010   Recoding
00001010
000011010010 Result
```

BOOTH MULTIPLICATION

Wallace Tree

در این معماری ی بیت های ضرب جزئی در هر ستون با توجه به مجموعه ای از شمارنده ها به صورت موازی بدون انتقال رقمی نقلی با یکدیگر جهمممع می شوند

